



Avsluttende eksamen i
TDT4125 ALGORITMEKONSTRUKSJON, VIDEREGÅENDE KURS

Lørdag 24. mai, 2014
Tid: 09:00–13:00

Hjelpemidler: (B) Alle trykte/håndskrevne; spesifikk, enkel kalkulator

Språk: Norsk (bokmål)

Les hele eksamen før du begynner, disponer tiden, og forbered spørsmål til fagstabens
hjelperunde. Gjør antakelser der det er nødvendig. Svar kort og konsist. Lange forklaringer
som ikke direkte besvarer spørsmålet gis ingen vekt.

Alle deloppgaver teller likt.

Oppgave 1

- a) Diskuter kort ideen med “slakking” (*relaxation*), slik det brukes om mengden med lovlige (*feasible*) løsninger. Hvordan kunne samme konsept brukes om mengden med lovlige problem-instansar? Hva ville forskjellen være? Hva er forholdet mellom denne formen for slakking og reduksjonar?
- b) Hvordan bruker man slakking når man lager approksimasjonsalgoritmer?
- c) Når man jobber med lineærprogrammering kan en likefrem bruk av slakking av og til føre til at man mister for mye struktur eller informasjon. Diskuter kort en vanlig metode for å gjøre dette mindre problematisk, og for å beholde noe informasjon om det en har fjernet, men i en mindre streng form.
- d) Hvordan kan man trygt fjerne tilfeldighetene fra randomiserte approksimeringsalgoritmer?
- e) Anta at du vil minimere målfunksjonen $\max_{i=1 \dots n} c_i x_i$, der koeffisientene c_i alle er forskjellige, og alle er positive heltll, og du har beskrankningen $x_i \in \{0, 1\}$, for $i = 1 \dots n$. Vis korleis du kunne redusere dette til målet om å minimere $\sum_{i=1}^n c'_i x_i$, med et annet sett koeffisientr c' , så den resulterande løsningen for x vil være optimal for det originale problemet ditt, uavhengig av andre beskrankninger.

Betrakt følgende heltallsprogram for TSP. Grafen $G = (V, E)$ har noder som samsvarer med byene og kanter som samsvarar med avstandene mellom dem. La $\delta(S)$ være delmengden av kantene i E som har nøyaktig ett endepunkt i S . Variabelen x_e i heltallsformuleringen er 1 om kanten er med i rundturen og 0 ellers.

$$\begin{aligned} \min \quad & \sum_{e \in E} c_e x_e \\ \text{s.t.} \quad & \sum_{e \in \delta(S)} x_e \geq 2, \quad \forall S \subset V, S \neq \emptyset, \\ & x_e \in \{0, 1\}, \quad \forall e \in E. \end{aligned}$$

- f) Selv om du ikke har noen garantier for kvaliteten på løsningen så vil du kjøre noen eksperimenter med en LP-slakking (*linear programming relaxation*) av programmet. Forklar hvordan du ville gjøre dette.

Min-max-TSP er en versjon av TSP der du ikke vil minimere *summen* av avstandane i rundturen. I stedet vil du minimere størrelsen på den største av disse avstandane. Med andre ord så vil vi minimere

$$\max \{c_{k(n)k(1)}, c_{k(1)k(2)}, \dots, c_{k(n-1)k(n)}\},$$

der c_{ij} er avstanden mellom byene i og j og $k(i)$ er den i -ende byen i rundturen.

- g) Skriv om heltallsprogrammet for TSP slik at det blir et heltallsprogram for min-max-TSP. Merk at metoden fra e) ikke vil fungere her.
- h) Vis at min-max-TSP er NP-hardt, selv om kostnaden c er metrisk, det vil si, $c_{ik} \leq c_{ij} + c_{jk}$.
- i) Hvilken approksimeringsfaktor ville dobbelt-tre-algoritmen (*the double-tree algorithm*) gi for min-max-TSP for metriske avstander? Forklar klart og grundig.

Betrakt nå såkalte *ultrametriske* avstander. Dette betyr at kostnadene nå følger den såkalte *sterke* trekantulikheten:

$$c_{ik} \leq \max\{c_{ij}, c_{jk}\}$$

Anta at du kan endre Christofides' algoritme for min-max-problemet på følgende måte: Du bruker noen innkapslede (*black-box*) prosedyrer for å effektivt finne både et min-max-spenntre (et spenntre som har en maksimal kant som er minimal over mengden med spenntreer) og en min-max-matching (en matching som har en maksimal kant som er minimal over mengden med matchinger). Du bruker nå disse i stedet for et minimalt spenntre og en min-cost matching i Christofides' algoritme.

- j) Hvilken approksimeringsfaktor ville denne endrede versjonen av Christofides' algoritme gi for min-max-TSP for ultrametriske avstander? Forklar klart og grundig.